

Don't Gobblefunk around with Words!*

Introduction to Cryptography: Appendix
Year 11 Profile Course with Ms. Fernandez and Josh

November 15, 2017

This document serves to elucidate some of the various concepts first described in our first lesson in a bit more detail and with some examples. Some things are repeated or described anew with slightly different words or context. As always feel free to mark up this document and take as much time as you need to get yourself to a point where you feel comfortable and confident with the material. This is about mastery, not speed. If you find that something is missing or you don't get something, go ahead and check back with your original sheet and then just come back here—don't worry, I'm not going anywhere!

As we mentioned, the most basic cryptography has to do with transforming *plaintext*, or normal readable words, into *ciphertext*, or words that one can (hopefully) only read if one knows a secret *key*, and the scheme with which one transforms ciphertext back to plaintext. Some schemes don't really have keys, but they are often not very secure in that they are easy crack (as we will see in the next example). As we discussed at the end of our first section, an encryption scheme is made up of at least two parts: an *encrypt*(·) function and a *decrypt*(·) function. Each of these takes a *word* as an argument or *input* and spits out another word as *output*. Let's go through a small example:

Suppose our pal Schlomo invented a code where the word *apple* can be encrypted to give us *aaappppppllleee* and *extemporaneous* can be encrypted to give us *eeexxttteemmmpppooorrraaannneeeooouuuss*. Hopefully the code is clear, and while we may question Schlomo's judgment it turns out this code can be super useful if we have a channel of communication that is prone to errors¹. The question is, how would we describe how that code works in a less ambiguous way? how would we write down the encrypt and decrypt functions!?

In order to write this code we would need to have a concept of function, and that is our actual goal over the next few weeks—learn functions through other cools stuff! The interesting thing about mathematics is that sometimes it takes more brain power to elucidate something obvious than something profound: in this case, we didn't even specify which alphabet Schlomo is using! We also didn't

*Roald Dahl, The Big Friendly Giant

¹this is called a Triple Modular Redundancy. The three characters "vote" and if one disagrees because of some kind of distortion in the signal, the odd one out is overruled, a great way to recover from errors though we will get to more error correction some other time perhaps—sometimes Schlomo is a smart cookie!

talk about how we build words and what kind of operations we are allowed to do with those building blocks!

The smallest building block we will be considering, the atoms of our cryptography universe, if you will, is that of characters: a character is just a symbol like any of the hundreds you are reading right now.² We call a set of characters in which we write our messages our *Alphabet*³. Furthermore, and this is critical, we usually index our alphabet, what does that mean? Indexing is just a fancy way of saying "put them in a specific order" or "count them" but it is one of the most powerful and tricky tools in mathematics!⁴ Furthermore, characters in our alphabet can be put together to make words, and these can have indices too! If you think you are clear with how indices work skip on ahead, otherwise, let's do a couple examples:

Let's look at a smaller alphabet $A = \{a, b, c, d, e\}$. What do words look like here? By words we don't necessarily mean words in English (or some other language) but rather combinations of the letters in our alphabet A . So a , b and all of the other letters themselves are words, as are ab , bc , $aaabbb$, $abede$, $bebe$, bae , $badcad$ and so on. In our hypothetical language $asdf$ is *not* a word, as Schlomo would look at that and ask "what are those things after the a and after the d ?"⁵ So ab , bc , $aaabbb$ etc. are all possible *plaintext* words in our language. We usually write the *language associated with* the alphabet A as $\mathcal{L}(A)$ but we won't use that for a while probably. As a last thing to words and alphabets: we often write the indices as little subscripts, but there are other ways of writing them as well, as long as you are consistent and you make it clear when you write (and you understand this and other related text) all is well. For example we might write a word w as

$$w = w_0 w_1 \cdots w_n$$

Where each of w_i (i for index—it took some getting used to for me) is a character. I started with 0, as per computer science convention, but no significant harm will come to the universe if you start at 1!⁶ For more practice we might get to use our "member of" symbol $\in$.

$$w = w_0 w_1 \cdots w_n \quad \text{with } w_i \in A$$

We can restrict our language $\mathcal{L}(A)$ as well as our alphabet as much as we want—we do this all the time in English: Shlammerworp is not an English word

²for those who now want me to define a symbol—the instinct is understandable, but we have to stop somewhere! Even in the deepest realms of mathematical logic we accept that some amount of prior knowledge needs to be there—in this case, that we know what symbols are, at least intuitively, and that we are beings who are able to get along quite well with such uncertainty.

³note that this is a classic example of where mathematics just claims something from our everyday life, calls it something more general, and suddenly it is holy: "a collection of symbols" is just what it sounds like, the next part is the kicker!

⁴some of the ugliest and also some of the most beautiful and clear math I have ever seen owes its failure or success to how indices were used—this is not a trivial matter and it took millennia for us to understand how to do this formally!

⁵just like how Anglophones get confused when reading things with Umlaute or Eszett... these things aren't in their alphabet! and besides, Anglophones just get confused often!

⁶Just please for the love of all things sacred don't start at 2!

(though it should be) even though it made of the letters of the alphabet. For our purposes though, most important is that you understand that when I have a word $w = abba$ that $w_0 = a$, $w_3 = a$ and the other two letters are b . We clear on that? If not, go ahead and do some examples for yourself!

Okay, lets go back to Schlomos example with the alphabet $A = \{a, b, c, d, e\}$ We could write our $encrypt(\cdot)$ and $decrypt(\cdot)$ functions as follows:

- $encrypt(w)$ takes in a word w from our language $\mathcal{L}(A)$ (based on our alphabet A) as its input and outputs a word (also in our language) where each character is tripled:

$$encrypt(w) \rightarrow encrypt(w_0w_1 \cdots w_n) \rightarrow w_0w_0w_0w_1w_1w_1 \cdots w_nw_nw_n$$

But we could write this prettier, lets just say that for each character $w_i \in w$ our encrypt function triples that character directly:

$$encrypt(w_i) = w_iw_iw_i \quad \text{for all } w_i \in w$$

See the power of indices?

- $decrypt(w)$ takes a word from our language $\mathcal{L}(A)$ (based on our alphabet A) as its input and outputs a word (also in our language) where each tripled character is put back to one, for each $w_iw_{i+1}w_{i+2} \in w$ (where $w = w_1w_2 \cdots w_n$) we have

$$decrypt(w_iw_{i+1}w_{i+2}) \rightarrow w_i \quad \text{if } w_i = w_{i+1} = w_{i+2}$$

So this is interesting, as now both of these functions seems to be going through every character in the word and either tripling them or taking the triplets and taking them back down to singletons like we expect. Everything seems fine right? But what happens if I try

$$decrypt(abcd)$$

what will happen? What should happen? On the other hand, $encrypt(\cdot)$ seems much less picky, if I run something through encrypt twice for example I just get something like this

$$encrypt(aba) \rightarrow aaabbbaaa$$

$$encrypt(encrypt(aba)) \rightarrow encrypt(aaabbbaaa) \rightarrow aaaaaaaaaabbbbbbbbaaaaaaaaaa$$

not super exciting, and super annoying to type, but seems to always work!

We will talk about this distinction more in the future, if you have any questions, feel free to ask—but first try your hand at the exercises! Sometimes the old noggin' takes time to figure out the stuff it's visual cortex just processed.